Selenium with Java: compact cheatsheet

STEP 1 Meet Selenium

1.1 What Selenium is and where it came from

What Selenium is An umbrella project of tools and libraries that automate real browsers the way a user would. At its core, a browser user agent library.

Three parts WebDriver (library plus browser driver, for code), IDE (record and play back extension), Grid (many sessions on many machines).

Not a test framework Selenium drives the browser. Assertions, test structure and reports come from JUnit, pytest, Mocha and similar tools.

Key dates Selenium 2.0 on 8 July 2011, 3.0 on 13 October 2016, W3C Recommendation on 5 June 2018, Selenium 4.0 on 13 October 2021.

Program shape Start a session, send the browser somewhere, ask or act, and always quit at the end.

```
WebDriver driver = new ChromeDriver();
```

Start Chrome. Selenium Manager finds a matching chromedriver for you, then a new browser session begins.

```
driver.get("https://www.selenium.dev/selenium/web/web-form.html");
```

Ask the browser to open the page and wait until it has loaded.

TRAP Selenium only controls the browser. It does not decide whether a test passed, it does not write reports and it does not give you Given, When, Then syntax. Those come from a test framework such as JUnit, pytest or Mocha. Chapter 5.1 joins the two.

EXAM Interviewers like to ask what changed in Selenium 4. The short answer: W3C protocol only, so capabilities must follow the W3C names (browserVersion, platformName) or carry a vendor prefix such as goog:chromeOptions. Options classes replaced DesiredCapabilities, Java lost the findElementById style helpers, and relative locators and Grid 4 arrived. Selenium Manager came later, in 4.6.

1.2 Architecture and the W3C WebDriver protocol

Three programs Your code and the binding (local end), the driver (remote end), the browser. The driver is written by the browser vendor.

HTTP and JSON Every command is one HTTP request with a fixed method and path. Answers are {"value": ...} on success or {"value": {"error", "message", "stacktrace"}} on failure.

Session id POST /session returns a session id. Every later path contains it. DELETE /session/{id} ends it.

Element reference Elements travel as {"element-6066-11e4-a52e-4f735466cecf": "id"}. Bindings turn id, name and class name into CSS selectors.

BiDi Classic WebDriver is request and response. WebDriver BiDi adds a WebSocket so the browser can send events. CDP access is temporary until BiDi covers it.

```
static HttpClient http = HttpClient.newHttpClient();
```

One HTTP client, reused for every command.

```
static String call(String method, String path, String body) throws Exception {
```

Send one W3C command and return the JSON answer as text.

```
HttpRequest.Builder req = HttpRequest.newBuilder(URI.create("http://localhost:9515" + path))
```

chromedriver listens on port 9515 unless told otherwise.

```
.header("Content-Type", "application/json");
```

The protocol speaks JSON in both directions.

```
String reply = http.send(req.build(), HttpResponse.BodyHandlers.ofString()).body();
```

Send it and wait for the answer.

```
static String value(String json, String key) {
```

A tiny helper that pulls one string out of the JSON answer.

```
Process driver = new ProcessBuilder("chromedriver", "--port=9515").start();
```

Start the driver program yourself, the job Selenium usually does for you.

```
Thread.sleep(1000);
```

Give chromedriver a moment to open its port.

```
String s = "/session/" + value(created, "sessionId");
```

Every later command names this session in its path.

```
call("POST", s + "/timeouts", "{\"implicit\":2000}");
```

Set Timeouts: let Find Element keep looking for up to 2 seconds, so the next page has time to load.

```
call("POST", s + "/url", "{\"url\":\"https://www.selenium.dev/selenium/web/web-form.html\"}");
```

Navigate To.

```
call("GET", s + "/title", null);
```

Get Title.

```
call("POST", s + "/element/" + box + "/value", "{\"text\":\"Selenium\"}");
```

Element Send Keys types into that element.

```
call("POST", s + "/element/" + button + "/click", "{}");
```

Element Click submits the form.

```
call("GET", s + "/element/" + message + "/text", null);
```

Get Element Text on the next page.

```
call("DELETE", s, null);
```

Delete Session closes Chrome.

```
driver.destroy();
```

Stop chromedriver.

Success HTTP 200 and {"value": result}

Error HTTP 4xx or 5xx and {"value": {"error", "message", "stacktrace"}}

1.3 Setup, Selenium Manager and your first script

Install Java: selenium-java with Maven or Gradle (Java 11+).

Selenium Manager Ships with Selenium since 4.6. A fallback that discovers the browser version, resolves, downloads and caches the driver in ~/.cache/selenium, and since 4.11 can download browsers.

Driver mismatch An old driver on PATH or given by hand gives session not created. Delete it and let Selenium Manager match the driver.

Eight components Start session, act on browser, ask the browser, wait strategy, find element, act on element, ask the element, end session.

```
String title = driver.getTitle();
```

3. Request browser information: the page title.

```
driver.manage().timeouts().implicitlyWait(Duration.ofMillis(500));
```

4. Establish a waiting strategy. Find calls will now retry for up to half a second.

```
WebElement textBox = driver.findElement(By.name("my-text"));
```

5. Find an element: the text box whose name attribute is my-text.

```
WebElement submitButton = driver.findElement(By.cssSelector("button"));
```

5. Find another: the first button on the page.

```
textBox.sendKeys("Selenium");
```

6. Take action on an element: type into the box.

```
submitButton.click();
```

6. Click Submit. The browser loads the next page.

```
WebElement message = driver.findElement(By.id("message"));
```

Find the message on the new page. The implicit wait gives it time to appear.

```
String value = message.getText();
```

7. Request element information: its visible text.

```
driver.findElement(By.id("no-such-id"));
```

This element does not exist, so this line throws.

TRAP If you pass a driver path yourself, Selenium trusts you and never calls Selenium Manager. The documentation says that with Selenium 4.6 or newer you should not need to set a driver location at all. Delete old drivers from PATH and let Selenium Manager do the job.

TRAP The documentation itself warns that an implicit wait is rarely the best solution. It is used here because it is the easiest to show. Chapter 4.1 replaces it with explicit waits.

STEP 2 Control the browser

2.1 Sessions, options and timeouts

Options Since Selenium 4, describe sessions with Options classes. They become W3C capabilities, and browser specific ones carry a prefix such as goog:chromeOptions.

Page load strategy normal waits for complete (load), eager for interactive (DOMContentLoaded), none for the first download. It covers navigation by address, not clicks.

Default timeouts Implicit 0, page load 300,000 ms, script 30,000 ms. Java prints PT0S, PT5M and PT30S.

Service Starts and stops a local driver: file, port, arguments and logs. Not used for remote sessions.

Granted capabilities Read them after New Session to learn the real browser version, platform and defaults such as dismiss and notify.

```
ChromeOptions options = new ChromeOptions();
```
An Options object describes the session you want. browserName is already set to chrome.

```
options.addArguments("--headless=new");
```
A Chrome command line switch: run without a visible window.

```
options.addArguments("--window-size=1280,800");
```
Another switch: the size of the browser window.

```
options.setPageLoadStrategy(PageLoadStrategy.EAGER);
```
A W3C capability: get() returns at DOMContentLoaded.

```
ChromeDriver driver = new ChromeDriver(options);
```
New Session is sent with these capabilities.

```
Capabilities caps = driver.getCapabilities();
```
The capabilities the driver actually granted.

```
WebDriver.Timeouts t = driver.manage().timeouts();
```
The timeouts of this session.

```
t.scriptTimeout(Duration.ofSeconds(1));
```
Allow scripts only one second from now on.

```
((JavascriptExecutor) driver).executeAsyncScript("/* never calls its callback */");
```
An async script must call its callback. This one never does.

```
ChromeDriverService service = new ChromeDriverService.Builder().usingPort(9600).build();
```
A Service describes how to start chromedriver: here, on port 9600.

```
ChromeDriver driver = new ChromeDriver(service);
```
Selenium starts chromedriver on that port, then opens a session.

```
HttpRequest status = HttpRequest.newBuilder(URI.create("http://localhost:9600/status")).build();
```
GET /status is a W3C command that needs no session.

Page load 300,000 ms

TRAP Switching to eager or none to make tests faster is a trade. Every element that arrives later now needs an explicit wait, or the test becomes flaky. Chapter 4.1 shows how to write those waits.

EXAM These timeouts belong to the browser session. Explicit waits such as WebDriverWait are different: they run in your code and poll the browser. Mixing an implicit wait with explicit waits can give surprising wait times, which chapter 4.1 explains.

2.2 Navigation, window size, screenshots and PDF

Navigation get or navigate().to, back, forward, refresh.

Where am I Title and current URL. Check them when a test depends on a redirect or on history, because back on the first page raises no error.

Window rect Set and get size and position. An exact size is more portable than maximize, which needs a window manager.

Screenshots Take Screenshot (viewport) and Take Element Screenshot return base64 PNG. Save one when a test fails, before quit.

Print Page Returns a base64 PDF. Use PrintOptions for orientation, ranges, size, margins and scale. Chromium needs headless.

```
driver.navigate().to("https://www.selenium.dev/selenium/web/xhtmlTest.html");
```
The long form does the same thing.

```
driver.navigate().back();
```
Press the Back button.

```
driver.navigate().forward();
```
Press the Forward button.

```
driver.navigate().refresh();
```
Reload the current page.

```
WebDriver.Window window = driver.manage().window();
```
Every window command lives under manage().window().

```
window.setSize(new Dimension(1024, 768));
```
Restore the window and make it 1024 by 768 pixels.

```
window.setPosition(new Point(40, 30));
```
Move its top left corner to x 40, y 30 on the screen.

```
Dimension size = window.getSize();
```
Ask the browser for the size it really has.

```
File page = ((TakesScreenshot) driver).getScreenshotAs(OutputType.FILE);
```
Take Screenshot of the viewport. Selenium writes a temporary PNG file.

```
Files.copy(page.toPath(), Path.of("page.png"));
```
Copy it somewhere you will find it.

```
File only = button.getScreenshotAs(OutputType.FILE);
```
Take Element Screenshot: just the area of the button.

```
BufferedImage img = ImageIO.read(new File(name));
```
Open the PNG to read its size in pixels.

```
ChromeOptions options = new ChromeOptions().addArguments("--headless=new");
```
Chromium prints to PDF only in headless mode.

```
PrintOptions print = new PrintOptions();
```
Paper settings: orientation, page ranges, size, margins, scale, background.

```
Pdf pdf = ((PrintsPage) driver).print(print);
```
The W3C Print Page command. The PDF comes back as base64 text.

```
byte[] bytes = Base64.getDecoder().decode(pdf.getContent());
```
Decode it to the real bytes.

TRAP After refresh or any navigation, the elements you found before belong to the old page. Using them raises stale element reference. Find them again on the new page. Chapter 5.4 covers this error in depth.

TRAP Maximize and minimize need a desktop window manager. On a bare Linux build server they can fail with an unknown error, which is what happened when this page was built. Setting an exact size with Set Window Rect or the --window-size switch works everywhere and gives the same layout on every machine.

STEP 3 Find and use elements

3.1 Locators: how Selenium finds an element

Eight strategies id, name, class name, css selector, xpath, tag name, link text, partial link text. find element returns the first match.

Five on the wire W3C defines css selector, link text, partial link text, tag name and xpath. Bindings turn id, name and class name into CSS.

CSS and XPath CSS: #id, .class, tag[attr="v"], parent > child. XPath adds text matching and moving up to a parent. Prefer relative XPath (//).

Relative locators above, below, toLeftOf, toRightOf and near (50 px), built on getBoundingClientRect. They depend on layout, so check what they return.

Preference Unique stable id, then a short CSS selector, then XPath. Avoid absolute paths, styling classes and generated ids. Consider data-test attributes.

```
static void show(String how, WebElement e) {
```
A helper that prints which element a locator found.

```
show("className", driver.findElement(By.className("information")));
```
The first element whose class list contains information.

```
show("cssSelector", driver.findElement(By.cssSelector("#fname")));
```
CSS: # means id, so the input with id fname.

```
show("id", driver.findElement(By.id("lname")));
```
The element whose id attribute is lname.

```
show("name", driver.findElement(By.name("newsletter")));
```
The element whose name attribute is newsletter.

```
show("linkText", driver.findElement(By.linkText("Selenium Official Page")));
```
A link whose whole visible text matches.

```
show("partialLinkText", driver.findElement(By.partialLinkText("Official")));
```
A link whose visible text contains Official.

```
show("tagName", driver.findElement(By.tagName("a")));
```
The first a element on the page.

```
show("xpath", driver.findElement(By.xpath("//input[@value='f']")));
```
XPath: any input whose value attribute is f.

```
show("above lname", driver.findElement(with(By.tagName("input")).above(By.id("lname"))));
```
An input above the lname box.

```
show("left of password", driver.findElement(with(By.tagName("input")).toLeftOf(By.id("password-field"))));
```
On the login page the three inputs sit in one row.

TRAP Each relative locator sends the page a script to run, so it is slower than a plain CSS selector. Prefer an id or a short CSS selector, and keep relative locators for elements with no good attributes.

EXAM Keep locators in one place, apart from the steps that use them. The documentation recommends declaring them separately, which is exactly what a page object does. Chapter 5.2 shows the pattern.

3.2 Finding elements: one, many, scoped and in shadow DOM

One or many find element returns the first match or raises no such element. find elements returns all matches or an empty list.

Search context You can search from the driver, from an element or from a shadow root. CSS from an element stays inside it.

XPath trap An XPath starting with // always starts at the document, even when called on an element. Use .// to search below the element.

Shadow DOM Find the host, get its shadow root, then search with a CSS selector. Chrome refuses XPath there.

Active element switchTo().activeElement() returns the focused element. Use it to test keyboard flow such as Tab order.

```
List<WebElement> inputs =
driver.findElements(By.tagName("input"))
;
```
Plural: a List of every input, in page order.

```
for (WebElement e : inputs) {
```
Walk the list like any Java list.

```
List<WebElement> none =
driver.findElements(By.id("email"));
```
No match: an empty list, never an error.

```
driver.findElement(By.id("email"));
```
Singular with no match: an exception.

```
WebElement form =
driver.findElement(By.tagName("form"));
```
First find the parent element.

```
WebElement last =
form.findElement(By.id("lname"));
```
Two commands: find the form, then find inside it.

```
WebElement same =
driver.findElement(By.cssSelector("form
#lname"));
```
The optimised version: one CSS selector, one command.

```
WebElement host =
driver.findElement(By.cssSelector("custo
m-checkbox-element"));
```
The shadow host: the element the hidden tree is attached to.

```
SearchContext shadowRoot =
host.getShadowRoot();
```
Get Element Shadow Root. A SearchContext, like the driver and an element.

```
WebElement checkbox =
shadowRoot.findElement(By.cssSelector(bo
x));
```
Search inside the shadow tree.

```
shadowRoot.findElement(By.xpath(".//inpu
t"));
```
XPath inside a shadow root.

```
driver.findElement(By.name("my-
text")).click();
```
Clicking a text box gives it the focus.

```
driver.switchTo().activeElement().sendKe
ys(Keys.TAB);
```
Press Tab on whatever has the focus, as a user would.

`TRAP` find elements never waits for the first element to appear unless an implicit wait is set, and then it returns as soon as at least one matches. An empty list can mean the page is not ready yet. Chapter 4.1 shows how to wait properly.

3.3 Interacting with elements and reading them

Five commands click, send keys, clear, submit and select. Prefer clicking the submit button over submit(). send keys appends, so clear first.

Built in checks Before acting the driver scrolls the element into view and checks it is interactable. Errors: element not interactable, invalid element state, element click intercepted.

Attribute vs property DOM attribute is the HTML, DOM property is the live value. The classic getAttribute mixes both.

Select and upload Select works only on select elements: by visible text, value or index. Upload by sending the full path to input type="file".

```
driver.manage().timeouts().implicitlyWai
t(Duration.ofSeconds(2));
```
As in chapter 1.3: let finds retry for up to 2 seconds, so the page after Submit has time to load.

```
text.sendKeys("first try");
```
Element Send Keys: types into the box, adding to what is already there.

```
text.clear();
```
Element Clear: empties the box.

```
text.sendKeys("Selenium");
```
Type the real value.

```
check2.click();
```
Element Click: ticks it, as a mouse click would.

```
driver.findElement(By.id("my-radio-
2")).click();
```
Clicking the second radio button unselects the first one.

```
driver.findElement(By.cssSelector("butto
n")).click();
```
Click Submit. The documentation prefers this to the submit() method.

```
static void attempt(String what,
Runnable step) {
```
Run one step and report the error it raises, if any.

```
attempt("type into disabled", () →
driver.findElement(By.name("my-
disabled")).sendKeys("x"));
```
A disabled input cannot take keyboard input.

```
attempt("clear readonly", () →
driver.findElement(By.name("my-
readonly")).clear());
```
A readonly input cannot be reset.

```
((JavascriptExecutor)
driver).executeScript(
```
Add a cookie banner that covers the bottom of the page, as many real sites do.

```
attempt("click under banner", () →
driver.findElement(By.cssSelector("butto
n")).click());
```
The banner sits over the centre of the button.

```
text.sendKeys("hello");
```
Type, so the live value differs from the HTML.

```
Select single = new
Select(driver.findElement(By.name("selec
tomatic")));
```
Wrap a select element in a Select object.

```
single.selectByVisibleText("Four");
```
Pick by the text the user sees.

```
single.selectByValue("two");
```
Pick by the value attribute.

```
single.selectByIndex(0);
```
Pick by position, counting from 0.

```
Select multi = new
Select(driver.findElement(By.id("multi")
));
```
A select with the multiple attribute.

```
multi.deselectByValue("eggs");
```
Only multiple lists can deselect.

```
Path file =
Files.writeString(Path.of("report.txt"),
"quarterly numbers");
```
Make a small file to upload.

```
WebElement input =
driver.findElement(By.name("my-file"));
```
The input with type="file".

```
input.sendKeys(file.toAbsolutePath().toS
tring());
```
Send the full path. No dialog opens, the file is simply attached.

STEP 4 Synchronise and control

4.1 Waiting strategies: implicit, explicit and fluent

Implicit wait A session setting, default 0. Every find keeps looking up to that long and returns as soon as the element appears. It only waits for presence.

Explicit wait A loop with a condition, timeout and polling interval. Defaults: Java 500 ms.

Expected conditions presence, visibility, clickable, text present, title contains, staleness. Wait for visibility before typing.

Never mix Implicit plus explicit makes waits unpredictable: 10 s with 15 s can time out after about 20 s. Keep implicit at 0 and use explicit waits.

```
driver.findElement(By.id("adder")).click
();
```
The page adds a red box, but only 1 second after the click.

```
driver.findElement(By.id("box0"));
```
Our code asks at once. No wait is set, so the driver checks only once.

```
WebElement box =
driver.findElement(By.id("box0"));
```
Not there yet, so the driver keeps checking until the box appears.

```
WebElement revealed =
driver.findElement(By.id("revealed"));
```
The input is in the page already, just hidden.

```
driver.findElement(By.id("reveal")).clic
k();
```
The page shows it 1 second later.

```
WebDriverWait wait = new
WebDriverWait(driver,
Duration.ofSeconds(2));
```
Wait at most 2 seconds. Java checks every 500 ms.

```
wait.until(d → revealed.isDisplayed());
```
A lambda: keep calling it until it returns true.

```
revealed.sendKeys("Displayed");
```
Safe now: the input is displayed, so it can take keys.

```
WebElement box =
wait.until(ExpectedConditions.presenceOf
ElementLocated(By.id("box0")));
```
A ready made condition. It returns the element it waited for.

```
WebElement input =
wait.until(ExpectedConditions.visibility
OfElementLocated(By.id("revealed")));
```
Present is not enough for typing. Wait until it is visible.

```
new WebDriverWait(driver,
Duration.ofSeconds(1)).withMessage("the
second box never came")
```
Our own message makes the failure readable.

```
.until(ExpectedConditions.presenceOfElem
entLocated(By.id("box1")));
```
Nobody clicked again, so box1 never appears.

```
Wait<WebDriver> wait = new
FluentWait<WebDriver>(driver)
```
FluentWait lets you set every part of the loop.

```
.withTimeout(Duration.ofSeconds(2))
```
Give up after 2 seconds.

```
.pollingEvery(Duration.ofMillis(300))
```
Check every 300 ms instead of 500.

```
.ignoring(ElementNotInteractableExceptio
n.class);
```
Treat this error as not yet and try again.

```
wait.until(d → {
```
The condition does the action itself and returns true when it worked.

4.2 Windows, tabs, frames and alerts

Windows Each window or tab has a handle. Save the original, switch to the new one, and switch back after close(). newWindow("tab") opens and switches in one step.

Frames Switch into an iframe by element, name or id, or index before finding inside it. defaultContent goes to the top, parentFrame up one level.

Title in frames Get Title always reports the top level page, even from inside a frame.

Alerts alert, confirm and prompt: getText, accept, dismiss, sendKeys. Wait with alert is present when it appears late.

Forgotten pop ups unhandledPromptBehavior defaults to dismiss and notify: the pop up is closed and the next command raises unexpected alert open.

```
String original =
driver.getWindowHandle();
```
The id of the current window. It stays the same for the whole session.

```
driver.findElement(By.name("windowTwo"))
.click();
```
This link has target="_blank", so it opens a new window.

```
new WebDriverWait(driver,
Duration.ofSeconds(2)).until(ExpectedCon
ditions.numberOfWindowsToBe(2));
```
Wait until the second window exists.

```
for (String handle :
driver.getWindowHandles()) {
```
Every open window, in the order they were opened.

```
if (!handle.equals(original))
driver.switchTo().window(handle);
```
Switch to the one that is not the original.

```
driver.close();
```
Close only this window.

```
driver.switchTo().window(original);
```
Switch back, or the next command fails with no such window.

```
driver.switchTo().newWindow(WindowType.T
AB);
```
Selenium 4: open a new tab and switch to it in one step.

```
driver.switchTo().frame(frame);
```
1. Switch with a WebElement: the most flexible way.

```
driver.switchTo().defaultContent();
```
Leave the frame: back to the top document.

```
driver.switchTo().frame("iframe1-name");
```
2. Switch with the frame name or id.

```
driver.switchTo().frame(0);
```
3. Switch with an index: the first frame of the page.

```
driver.findElement(By.id("alert")).click
();
```
The page calls alert('cheese').

```
Alert alert =
wait.until(ExpectedConditions.alertIsPre
sent());
```
Wait for the alert, then get a handle on it.

```
alert.accept();
```
Press OK.

```
driver.findElement(By.id("confirm")).cli
ck();
```
A confirm box: OK or Cancel.

```
Alert confirm =
driver.switchTo().alert();
```
Switch to it without waiting, because it opens at once.

```
confirm.dismiss();
```
Press Cancel.

```
driver.findElement(By.id("prompt")).clic
k();
```
A prompt asks for text.

```
prompt.sendKeys("Selenium");
```
Type the answer into the prompt.

```
driver.findElement(By.id("slow-
alert")).click();
```
This one opens 200 ms later.

```
driver.getTitle();
```
Any command while an alert is open.

TRAP After close() you are still pointing at the closed window. The documentation warns that the next command raises no such window. Always switch back to a handle you saved before.

4.3 The Actions API: mouse, keyboard and wheel

Build then perform An action chain is sent in one request when perform() is called. Nothing happens before that.

Ticks Each input source has one action per tick. A tick lasts as long as its slowest action.

Mouse doubleClick, contextClick, moveToElement, clickAndHold, dragAndDrop, moveByOffset. The driver remembers the pointer position between chains.

Keyboard keyDown and keyUp for modifiers, sendKeys for text. Use Command on a Mac and Control elsewhere. Release every key you press.

Wheel scrollToElement, scrollByAmount, scrollFromOrigin. Wait for the scroll result, because it can land after perform() returns.

```
new
Actions(driver).doubleClick(box).perform
();
```
Build a chain, then perform() sends it to the browser in one request.

```
new
Actions(driver).contextClick(box).perfor
m();
```
A right click.

```
new
Actions(driver).moveToElement(driver.fin
dElement(By.id("hover"))).perform();
```
Move to the centre of the element. Nothing is clicked.

```
new Actions(driver).dragAndDrop(from,
to).perform();
```
Move to from, press, move to to, release.

```
new Actions(driver).moveToLocation(8,
12).perform();
```
A point measured from the top left of the viewport.

```
new Actions(driver).moveByOffset(13,
15).perform();
```
From wherever the pointer is now. The driver remembered it between chains.

```
.keyDown(Keys.SHIFT)
```
Press Shift and keep holding it.

```
.sendKeys("abc")
```
Goes to the element that has focus. The page focuses this box on load.

```
.keyUp(Keys.SHIFT)
```
Let go of Shift.

```
Keys cmdCtrl =
Platform.getCurrent().is(Platform.MAC) ?
Keys.COMMAND : Keys.CONTROL;
```
Select all is Command A on a Mac and Control A elsewhere.

```
new Actions(driver).sendKeys(input,
"Selenium!").perform();
```
Clicks the element first, then types.

```
WebElement frame =
driver.findElement(By.tagName("iframe"))
;
```
The page starts with a 5000 px tall block, so the iframe is far below.

```
new
Actions(driver).scrollToElement(frame).p
erform();
```
Scroll until the bottom of the element is in view.

```
new Actions(driver).scrollByAmount(0,
300).perform();
```
Like turning the wheel: 300 px down from the top left of the viewport.

```
new
Actions(driver).scrollFromOrigin(WheelIn
put.ScrollOrigin.fromElement(inner), 0,
200).perform();
```
Turn the wheel with the pointer over the frame, so the frame scrolls.

TRAP Actions type into the focused element. If nothing has focus, the keys go to the page body and the test seems to type into nothing. Element sendKeys focuses the element itself, which is why it is the better choice for plain typing.

4.4 JavaScript execution and cookies

executeScript The script runs as a function body. Use return to get a value and arguments[n] to read what you pass in. Elements go in and come back as WebElements.

Return types Whole numbers are Long in Java and int in Python, objects are Map and dict, undefined is null or None.

Async scripts executeAsyncScript adds a callback as the last argument. A returned promise is also awaited. The script timeout, 30 s by default, limits both.

Cookies Visit the domain first, then add. httpOnly hides a cookie from page scripts. deleteAllCookies gives a clean start.

```
static String type(Object o) { return o
== null ? "null" :
o.getClass().getSimpleName(); }
```
The Java class of whatever came back.

```
JavascriptExecutor js =
(JavascriptExecutor) driver;
```
Every driver class implements JavascriptExecutor, so this cast is safe.

```
js.executeScript("arguments[0].click();"
,
driver.findElement(By.id("updatediv")));
```
A click done by JavaScript. It skips the checks a real click makes.

```
driver.manage().timeouts().scriptTimeout
(Duration.ofMillis(200));
```
How long a script may run. The default is 30 seconds.

```
driver.manage().addCookie(new
Cookie("key", "value"));
```
The browser is still on an empty start page with no domain.

```
driver.get("https://www.selenium.dev/sel
enium/web/blank.html");
```
Visit the domain first. A small page is enough.

```
Cookie c =
driver.manage().getCookieNamed("key");
```
Read one cookie by name.

```
driver.manage().addCookie(new
Cookie.Builder("foo",
"bar").sameSite("Strict").build());
```
The builder sets the other fields.

```
driver.manage().addCookie(new
Cookie.Builder("secret",
"s").isHttpOnly(true).build());
```
httpOnly hides the cookie from page JavaScript.

```
driver.manage().deleteAllCookies();
```
A clean start, for example before logging in as another user.

STEP 5 Build a test suite

5.1 Test frameworks: JUnit 5, pytest and Mocha

Framework jobs Selenium drives the browser. JUnit 5, pytest or Mocha find the tests, run setup and teardown, assert and report.

Fresh browser Start the driver in the before each step and quit it in the after each step, so tests stay independent and can run in any order.

Teardown always runs @AfterEach, code after yield and afterEach run also when the test failed, so quit belongs there.

Screenshot on failure Take it after the test failed and before quit: a JUnit AfterTestExecutionCallback.

```
WebDriver driver;
```
A field, so every method of this test can reach the driver.

```
@BeforeEach
```
Runs before every test method.

```
@Test
```
One test: one behaviour, one reason to fail.

```
assertEquals("Web form",
driver.getTitle());
```
Expected value first in JUnit, then the actual one.

```
@AfterEach
```
Runs after every test, also when the test failed.

```
@ParameterizedTest(name = "{0} / {1}")
```
The name shows the data of each run in the report.

```
@CsvSource({
```
Each line is one run of the test: username, password, expected message.

```
void login(String user, String password,
String expected) {
```
JUnit passes the columns in as parameters.

```
Alert alert = new WebDriverWait(driver,
Duration.ofSeconds(2)).until(ExpectedCon
ditions.alertIsPresent());
```
The page answers with an alert.

```
if
(context.getExecutionException().isPrese
nt()) {
```
Present only when the test threw, so only when it failed.

```
Path target = Path.of("failure-" +
context.getRequiredTestMethod().getName(
) + ".png");
```
Name the file after the test.

```
@ExtendWith(ScreenshotOnFailure.class)
```
Plug the extension into this test class.

```
assertEquals("Web form!",
driver.getTitle());
```
Wrong on purpose, so the test fails.

5.2 Page objects and page components

Page object A class that is the interface to one page. Tests call its methods, so a change to the page is fixed in one place.

No assertions inside Assertions belong to the test. The only check in a page object is that the right page is loaded.

Return the next page A method that moves to another page returns that page object. A method that stays may return itself.

Components A page component object models a repeated part, such as a field or a product card, and searches only inside its root element.

Waits inside Put waits in the page object, for example a constructor that waits for its title after a navigation.

```
private final By fieldsBy =
By.cssSelector("label.form-label");
```
Locators live here and nowhere else.

```
if (!"Web
form".equals(driver.getTitle())) {
```
The one check a page object should make: am I on the right page?

```
return
driver.findElements(fieldsBy).stream().m
ap(Field::new).toList();
```
Wrap every label element in a Field component.

```
public String name() { return
root.getText().split("\n")[0].trim(); }
```
The first line of the label is its name.

```
private WebElement control() { return
root.findElement(By.cssSelector("input,
textarea, select")); }
```
Search inside this component only.

```
return this;
```
Returning itself allows field(...).fill(...).fill(...).

```
new WebDriverWait(driver,
Duration.ofSeconds(3)).until(ExpectedCon
ditions.titleIs("Web form - target
page"));
```
The page arrives after a navigation, so wait for it instead of checking once.

```
WebFormPage form =
WebFormPage.open(driver);
```
The test talks to pages, not to locators.

```
ReceivedPage received = form.submit();
```
submit() moves to another page, so it returns that page object.

```
assertEquals("Received!",
received.message());
```
The assertion stays in the test.

```
assertEquals(11, fields.size());
```
Eleven labelled controls on this page.

```
assertThrows(IllegalStateException.class
, () → new WebFormPage(driver));
```
The constructor checks the page and fails loudly.

5.3 Good practices and things to avoid

Need a browser? Ask first whether a lighter test can do the job. Browser tests are the most expensive kind.

Short and focused Set up the data, perform a few actions, evaluate the result. Many small independent tests beat one grand journey.

Clean start A new driver starts with a new profile. Use a fresh browser per test and never share test data.

Prepare state outside the UI Log in and create data through APIs, then set cookies, instead of clicking through forms.

Avoid CAPTCHAs, download progress, status codes, third party logins, test order, performance numbers, spidering and 2FA are jobs for other tools or test hooks.

```
first.manage().addCookie(new
Cookie("cart", "3 items"));
```
State a first test leaves behind: a cookie

```
((JavascriptExecutor)
first).executeScript("localStorage.setIt
em('theme', 'dark')");
```
and a value in local storage.

```
WebDriver second = new ChromeDriver();
```
A new driver gets a new, empty profile by default.

```
String session = "abc123";
```
In a real suite this value comes from a login API call, not from the login form.

```
driver.get("https://www.selenium.dev/sel
enium/web/cookies.html");
```
Now open the page under test. It reads its cookies on load.

5.4 Troubleshooting: errors, stale elements and logs

Error names Each W3C error has a class per language. Read the name, then the message, then your line in the trace.

Stale element The node behind a reference was replaced, for example by a refresh. Find the element again.

Context An element found in one frame or window is unknown in another. Switch back before using it.

Client log Java: java.util.logging at FINE.

Driver log ChromeDriver writes COMMAND and RESPONSE lines when its service is given a log file and a verbose level.

```
WebElement box =
driver.findElement(By.name("my-text"));
```
The driver gives back a reference to this exact node.

```
box = driver.findElement(By.name("my-
text"));
```
The fix: find it again.

```
WebElement email =
driver.findElement(By.id("email"));
```
Found inside the frame.

```
driver.switchTo().frame("iframe1");
```
The fix: switch back to the context where it was found.

```
static final Logger HTTP =
Logger.getLogger("org.openqa.selenium.re
mote.http.jdk.JdkHttpClient");
```
Keep a reference, or Java may forget the level you set.

```
HTTP.setLevel(Level.FINE);
```
Java logs at INFO by default. Request lines are written at FINE.

```
HTTP.addHandler(new Handler() {
```
A tiny handler that prints one line per request.

```
String m = new
SimpleFormatter().formatMessage(r);
```
The record holds a pattern plus values, so format it first.

```
.withLogFile(log)
```
Where ChromeDriver writes its log.

```
.withLogLevel(ChromiumDriverLogLevel.DEB
UG)
```
More detail than the default.

```
if (line.contains("] COMMAND "))
System.out.println(line.split("] COMMAND
")[1].split(" ")[0]);
```
Print the name of each command the driver received.

STEP 6 Scale out with Grid

6.1 Selenium Grid 4

Why a Grid Parallel runs on many browsers, versions and systems, and shorter suites: tests × average time ÷ nodes.

Components Router (4444), New Session Queue, Distributor, Session Map, Node (5555) and Event Bus (4442, 4443).

Roles Standalone for one machine, Hub and Node for most Grids, Distributed for very large ones.

Remote driver Only the driver creation changes. se:name adds metadata, se:cdp gives DevTools through the Grid, and a local file detector sends upload files to the node.

Queue and quit New sessions wait in the queue for a free slot. Always quit, or the slot stays busy until the session timeout.

```
@SuppressWarnings("unchecked")
```

JSON gives back plain objects, so the casts are unchecked.

```
HttpRequest.newBuilder(URI.create("http://localhost:4444/status")).build(),
```

The same endpoint a driver has, now answered by the Grid.

```
Map<String, Object> body = new Json().toType(res.body(), Json.MAP_TYPE);
```

Selenium ships its own JSON reader.

```
ExecutorService pool = Executors.newFixedThreadPool(3);
```

Three threads, so three tests start at the same moment.

```
RemoteWebDriver driver = new RemoteWebDriver(new URL("http://localhost:4444"), new ChromeOptions());
```

Returns when the Grid has given this test a slot.

```
Thread.sleep(3000);
```

Pretend the test takes 3 seconds.

```
options.setCapability("se:name", "upload on the grid");
```

se: capabilities are Grid metadata. se:name shows in the Grid UI.

```
RemoteWebDriver driver = new RemoteWebDriver(new URL("http://localhost:4444"), options);
```

The only change from a local test: a URL instead of a local driver.

```
driver.setFileDetector(new LocalFileDetector());
```

Sends the local file to the node, which may be another machine.

Suite time on a Grid

$$\text{total time} = \frac{\text{number of tests} \times \text{average test time}}{\text{number of nodes}}$$

6.2 WebDriver BiDi, CDP and Grid features

BiDi A W3C protocol with a WebSocket, so the browser can push events. Turn it on with webSocketUrl, then subscribe to console and page errors.

Events, not polling Handlers are called when events arrive. Wait for them, because they come after the action that caused them.

CDP Chrome and Edge only, no stability promise, the three newest Chrome versions. Use it for Chromium features with no other API.

GraphQL POST a query to /graphql on the Grid for nodes, slots, sessions and the queue size.

Grid endpoints /status, DELETE /session, and drain or remove a node through the Distributor endpoints.

```
options.setCapability("webSocketUrl", true);
```

Ask for a BiDi WebSocket next to the normal HTTP connection.

```
List<String> heard = new CopyOnWriteArrayList<>();
```

Events arrive on another thread, so use a thread safe list.

```
try (LogInspector inspector = new LogInspector(driver)) {
```

Subscribes to log events. Closing it unsubscribes.

```
inspector.onConsoleEntry(e -> heard.add("console " + e.getLevel() + ": " + e.getText()));
```

Called for every console.log, console.error and so on.

```
inspector.onJavaScriptException(e -> heard.add("page error: " + e.getText()));
```

Called for every uncaught error in the page.

```
while (heard.size() < 3 && System.currentTimeMillis() < end) Thread.sleep(50);
```

Events are pushed, not returned, so wait until all three arrived.

```
ChromeDriver driver = new ChromeDriver();
```

executeCdpCommand exists on Chrome and Edge drivers only.

```
Map<String, Object> version = driver.executeCdpCommand("Browser.getVersion", Map.of());
```

Any CDP method by name, with a map of parameters.

```
driver.executeCdpCommand("Emulation.setTimezoneOverride", Map.of("timezoneId", "Asia/Kolkata"));
```

Something WebDriver Classic has no command for.

```
static final Json JSON = new Json();
```

Selenium ships its own JSON reader.

```
String body = "{\"query\": \"" + query + "\"}";
```

GraphQL over HTTP: one POST with the query as a string.

```
return (Map<String, Object>) reply.get("data");
```

The answer sits under data.

```
String q = "{ grid { nodeCount, totalSlots, maxSession, sessionCount, sessionQueueSize } }";
```

Ask only for these fields.

```
.map(x -> JSON.<Map<String, Object>>toType((String) x.get("capabilities"), Json.MAP_TYPE).get("se:name"))
```

The se:name metadata is in the session capabilities.

STEP 7 Revise and practise

7.1 Revision: predict, debug and design

Predict, then run The driver does what the protocol says, not what the code seems to say. Offsets add, Shift changes case, a refresh makes references stale.

Cause before fix Name why a test is slow or flaky before changing it. Sleeps, mixed waits and missing quit calls are the usual causes.

Layers Tests, page objects and components, a driver factory and support code. Each change should touch one layer.

Size with the formula Suite time = tests × average time ÷ nodes, with about one CPU and 1 GB of RAM per browser.

Know your binding Time is a Duration in Java, seconds in Python and milliseconds in JavaScript, and a missing cookie gives null, None or an error.

Explicit wait poll 500 ms

Implicit wait 0

TRAP A good bug answer has three parts: what the code really does, why that breaks, and a fix that removes the cause. A retry that hides the failure is not a fix.